```
  import esp32
  iahgh = True
else:
  iahgh = False

pild = "LED"
if -1 != pfrm.find('ESP32C3'):
  # onboard LED is invert
logic based on ESP32C3 Super
Mini
  pild = 8

iled = Signal(Pin(pild, Pin.
OUT), invert=iahgh)

ahgh = False
eleds = (
          Signal(Pin(0, Pin.
OUT), invert=ahgh),
          Signal(Pin(1, Pin.
OUT), invert=ahgh),
          Signal(Pin(3, Pin.
OUT), invert=ahgh),
        )

if -1 == pfrm.find('ESP32C3'):
  isnsr = ADC(4)
else:
  isnsr = None

esnsr = Pin(5)
esnsr = dht.DHT11(esnsr)

mq2ao = ADC(2)
if -1 != pfrm.find('ESP32C3'):
  mq2ao.atten(ADC.ATTN_11DB)

ss = TM1637(Pin(6), Pin(7))

def aledoff():
  iled.off()
  eleds[0].off()
  eleds[1].off()
  eleds[2].off()

def rledon():
  eleds[0].on()
  eleds[1].off()
  eleds[2].off()

def gledon():
  eleds[0].off()
  eleds[1].on()
  eleds[2].off()

def bledon():
  eleds[0].off()
  eleds[1].off()
  eleds[2].on()

def getietemphdt():
  iled.on()
  sleep(2)
  iled.off()

  if isnsr:
    conversion_factor = 3.3
/ 65535
    reading = isnsr.read_
u16() * conversion_factor
    # The temperature sensor
measures the Vbe voltage
of a biased bipolar diode,
connected to the fifth ADC
channel
    # Typically, Vbe = 0.706V
at 27 degrees C, with a slope
of -1.721mV (0.001721) per
degree.
    itemp = 27 - (reading -
0.706)/0.001721
  else:
    itemp = esp32.mcu_
temperature()

  try:
    esnsr.measure()
  except:
    aledoff()

  etemp = esnsr.temperature()
  hum   = esnsr.humidity()
  if -1 != pfrm.
find('ESP32C3'):
    gsra  = mq2ao.read()
    gsrv  = (mq2ao.read_
uv())/1000000
  else:
    gsra  = mq2ao.read_u16()
    gsrv  = gsra * (3.3 /
65535)


  if etemp <= 30.00:
    gledon()
  elif etemp > 30.00 and
etemp <= 35.00:
    bledon()
  elif etemp > 35.00:
    rledon()

  print("%.2f, %.2f,
%.2f, %d, %.2f"
%(itemp,etemp,hum,gsra,gsrv))
  ss.numbers(int(etemp),
int(hum))
  sleep(2)
  ss.number(gsra)

while True:
  try:
    getietemphdt()
  except:
    aledoff()
    if -1 == pfrm.
find('ESP32C3'):
      reset()

#--- End of tempnumgss.py ---
```

The assembled air quality device soldered on a small PCB is shown in Figure 7. Just add the networking code shown in the last article and you could access the monitored air quality data on your mobile or monitoring server as well. Now, with all the hands-on knowledge of both Pi Pico and Super Mini along with multiple highly affordable and useful sensor modules, you are well versed to create some remarkable IoT devices and their networked applications.

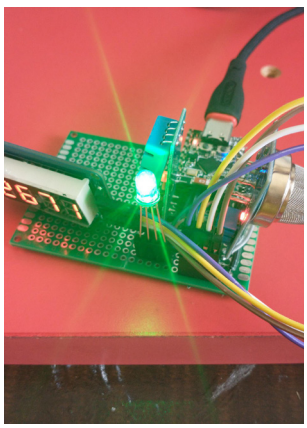


Figure 7: Super Mini-based air quality monitoring device

ESP32 C3 Super Mini is a compact, powerful and highly affordable IoT board you can use to create networked sensors. It fully supports MicroPython, so it can interface with tonnes of real world sensors and applications. It's an ideal companion to the Raspberry Pi ecosystem for creating cutting-edge products. END

References

- Esptool documentation, *https://docs.espressif.com/projects/esptool/en/latest/esp32/index.html*
- Micropython ESP32 quick references, *https://docs.micropython.org/en/latest/esp32/quickref.html*
- Micropython TM1637 library, *https://github.com/mcauser/micropython-tm1637*

By: Ankur Kumar

The author is a passionate researcher of free and open source software. He loves to explore cutting-edge technologies, ancient sciences, quantum spirituality, various genres of music, mystical literature, and art.